```
var name = ele.getAttribute('data-name');       ❸
</script>
```

❶ Write the value you want in JavaScript to a data-* attribute. This HTML-encodes the data.

❷ Gets a reference to the HTML element

❸ Reads the data-* attribute into JavaScript, which converts it to JavaScript encoding

XSS attacks are still common, and it's easy to expose yourself to them whenever you allow users to input data. Validation of the incoming data can help sometimes, but it's often a tricky problem. For example, a naive name validator might require that you use only letters, which would prevent most attacks. Unfortunately, that doesn't account for users with hyphens or apostrophes in their name, let alone users with non-Western names. People get (understandably) upset when you tell them that their name is invalid, so be wary of this approach!

Whether or not you use strict validation, you should always encode the data when you render it to the page. Think carefully whenever you find yourself writing `@Html.Raw()`. Is there any way, no matter how contrived, for a user to get malicious data into that field? If so, you'll need to find another way to display the data.

XSS vulnerabilities allow attackers to execute JavaScript on a user's browser. The next vulnerability we're going to consider lets them make requests to your API as though they're a different logged-in user, even when the user isn't using your app. Scared? I hope so!

29.2 Protecting from cross-site request forgery (CSRF) attacks